# IDA Pro MCP+Trae结合分析远古病毒熊猫烧香

昨夜西风凋碧树　发表于 北京　AI专栏　309浏览 · 2026-01-11 23:49

## 前言

```
现如今，各种恶意程序层出不穷，如何快速分析恶意程序的行为、节省安全从业人员的时间是一个问题。
幸好在当今大模型、智能体、MCP流行的时代，能够提供部分答案。

IDA Pro MCP 是一个AI驱动的反编译辅助工具，
通过MCP协议将IDA Pro与语言模型连接，旨在提升逆向工程效率。

它由三个主要组件组成：
IDA Pro 远程控制插件 (ida_remote_server.py)：一个IDA Pro插件，创建一个HTTP服务器以远程控制IDA Pro功能。
IDA 远程客户端 (idaremoteclient.ts)：一个TypeScript客户端，用于与IDA Pro远程控制服务器进行交互。
MCP 服务器 (index.ts)：一个模型上下文协议服务器，向AI助手暴露IDA Pro的功能。

功能
从AI助手执行IDA Pro中的Python脚本
检索二进制文件的信息：
字符串、导入、导出函数
高级二进制分析功能：
1、在指令中搜索立即数
2、在二进制文件中搜索文本字符串
3、搜索特定的字节序列
4、获取地址范围内的反汇编代码
5、通过标准化接口自动化IDA Pro操作
6、组件之间的安全通信
```

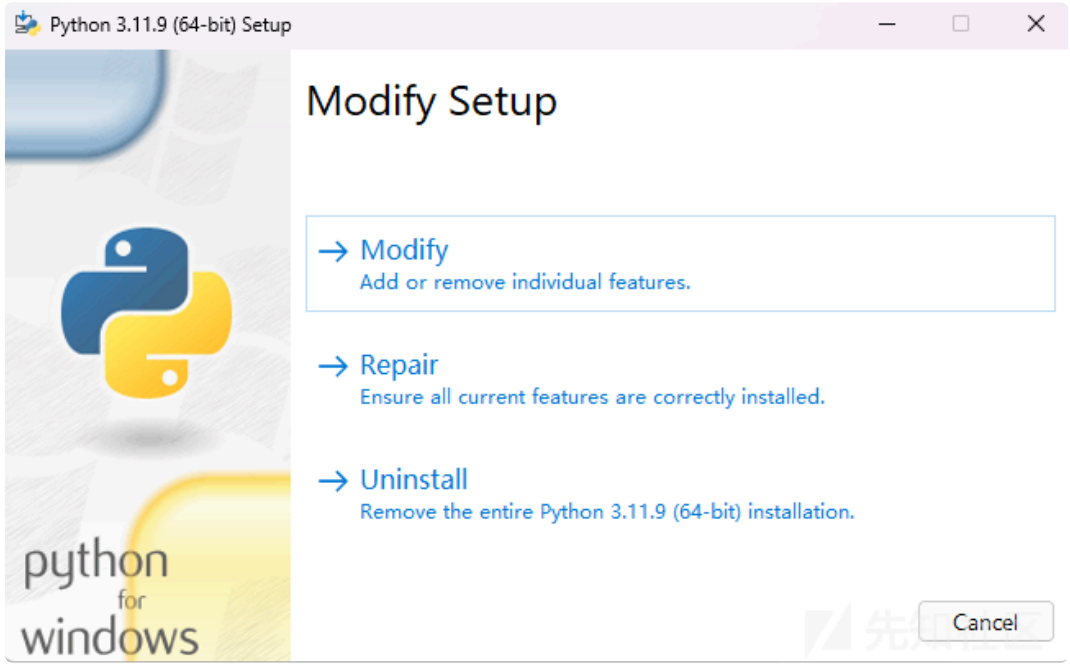
# 环境搭建

## Python环境

访问地址下载python 3.11.9版本。

下载地址：https://www.python.org/ftp/python/3.11.9/python-3.11.9-amd64.exe

下载好直接一键安装即可，这里是已经安装好了。所以显示这个界面

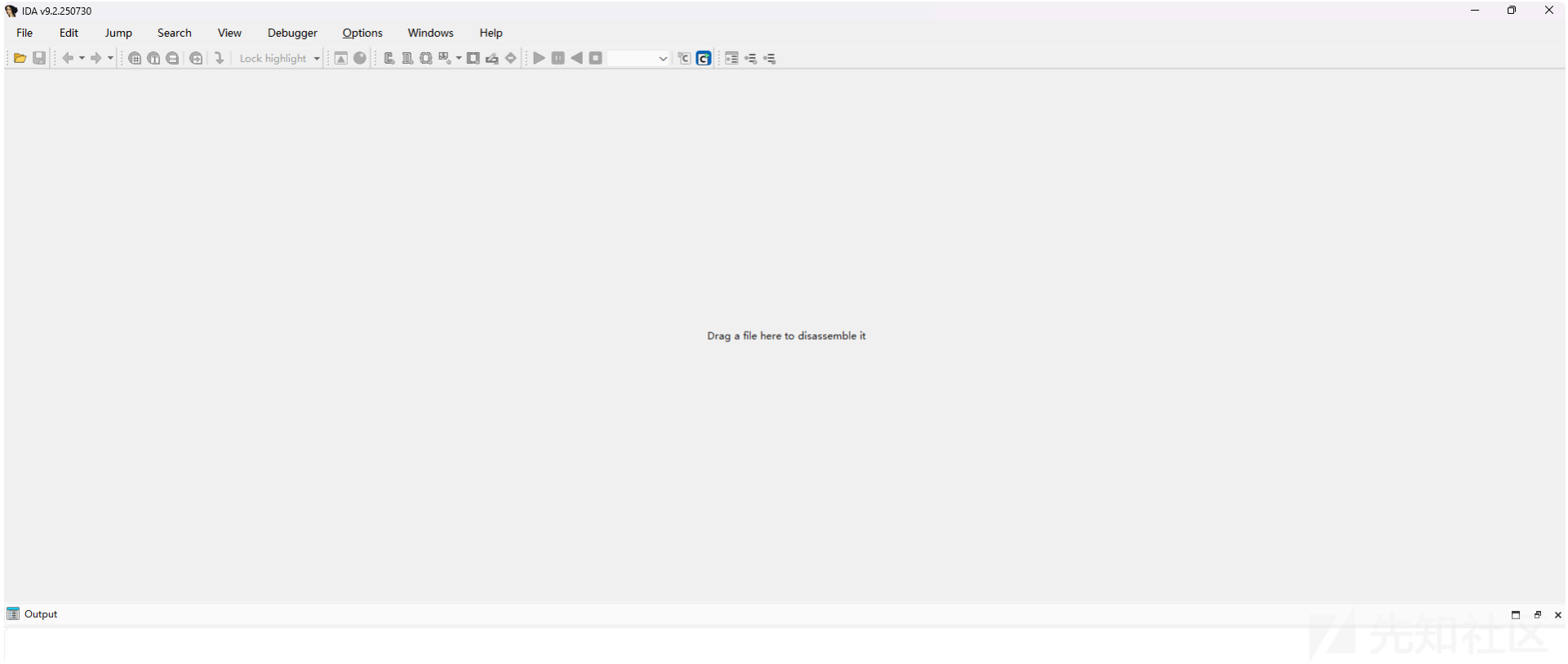



验证python是否安装成功

```
D:\>python
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

## IDA Pro MCP

下载好IDA Pro9.0版本以上，并安装好




使用如下命令安装mcp

```
pip install --upgrade git+https://github.com/mrexodia/ida-pro-mcp
```

```
D:\IDA92\python>pip install --upgrade git+https://github.com/mrexodia/ida-pro-mcp
Collecting git+https://github.com/mrexodia/ida-pro-mcp
  Cloning https://github.com/mrexodia/ida-pro-mcp to c:\users\orange\appdata\local\temp\pip-req-build-skxxi0jj
  Running command git clone --filter=blob:none --quiet https://github.com/mrexodia/ida-pro-mcp 'C:\Users\orange\AppDat
Local\Temp\pip-req-build-skxxi0jj'
  Resolved https://github.com/mrexodia/ida-pro-mcp to commit c2472d1c1e676f0198070e0e27f708973c7a4254
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting idapro>=0.0.7 (from ida-pro-mcp==2.0.0)
  Downloading idapro-0.0.7-py3-none-any.whl.metadata (6.7 kB)
Collecting tomli-w>=1.0.0 (from ida-pro-mcp==2.0.0)
  Downloading tomli_w-1.2.0-py3-none-any.whl.metadata (5.7 kB)
Downloading idapro-0.0.7-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 630.3 kB/s eta 0:00:00
Downloading tomli_w-1.2.0-py3-none-any.whl (6.7 kB)
Building wheels for collected packages: ida-pro-mcp
  Building wheel for ida-pro-mcp (pyproject.toml) ... done
  Created wheel for ida-pro-mcp: filename=ida_pro_mcp-2.0.0-py3-none-any.whl size=95403 sha256=8a60fb834de6c8ff048e9e2
a6317f6876129f1c51405359b707bdced3d6138
  Stored in directory: C:\Users\orange\AppData\Local\Temp\pip-ephem-wheel-cache-u6nhbdg4\wheels\93\ad\8f\7a5df63356b9e
0c9239927a65908bf010fc21cb52a8362c0
Successfully built ida-pro-mcp
Installing collected packages: tomli-w, idapro, ida-pro-mcp
Successfully installed ida-pro-mcp-2.0.0 idapro-0.0.7 tomli-w-1.2.0

[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

先知社区

运行以下命令，将MCP插件安装到IDA Pro

C 复制代码

```
"D:\Python311\Scripts\ida-pro-mcp.exe" --install
```

```
D:\IDA92\python>"D:\Python311\Scripts\ida-pro-mcp.exe" --install
Installed IDA Pro plugin (IDA restart required)
  loader: C:\Users\orange\AppData\Roaming\Hex-Rays\IDA Pro\plugins\ida_mcp.py
  package: C:\Users\orange\AppData\Roaming\Hex-Rays\IDA Pro\plugins\ida_mcp
Skipping Cline installation
  Config: C:\Users\orange\AppData\Roaming\Code\User\globalStorage\saoudrizwan.claude-dev\settings\cline_mcp_settings.json (not found)
Skipping Roo Code installation
  Config: C:\Users\orange\AppData\Roaming\Code\User\globalStorage\rooveterinaryinc.roo-cline\settings\mcp_settings.json (not found)
Skipping Kilo Code installation
  Config: C:\Users\orange\AppData\Roaming\Code\User\globalStorage\kilocode.kilo-code\settings\mcp_settings.json (not found)
Skipping Claude installation
  Config: C:\Users\orange\AppData\Roaming\Claude\claude_desktop_config.json (not found)
Installed Cursor MCP server (restart required)
  Config: C:\Users\orange\.cursor\mcp.json
Skipping Windsurf installation
  Config: C:\Users\orange\.codeium\windsurf\mcp_config.json (not found)
Installed Claude Code MCP server (restart required)
  Config: C:\Users\orange\.claude.json
Skipping LM Studio installation
  Config: C:\Users\orange\.lmstudio\mcp.json (not found)
Skipping Codex installation
  Config: C:\Users\orange\.codex\config.toml (not found)
Skipping Zed installation
  Config: C:\Users\orange\AppData\Roaming\Zed\settings.json (not found)
Skipping Gemini CLI installation
  Config: C:\Users\orange\.gemini\settings.json (not found)
Skipping Qwen Coder installation
  Config: C:\Users\orange\.qwen\settings.json (not found)
Skipping Copilot CLI installation
  Config: C:\Users\orange\.copilot\mcp-config.json (not found)
Installed Crush MCP server (restart required)
  Config: C:\Users\orange\crush.json
Installed Augment Code MCP server (restart required)
  Config: C:\Users\orange\AppData\Roaming\Code\User\settings.json
Installed Qodo Gen MCP server (restart required)
  Config: C:\Users\orange\AppData\Roaming\Code\User\settings.json
Skipping Antigravity IDE installation
  Config: C:\Users\orange\.gemini\antigravity\mcp_config.json (not found)
Skipping Warp installation
  Config: C:\Users\orange\.warp\mcp_config.json (not found)
Skipping Amazon Q installation
  Config: C:\Users\orange\.aws\amazonq\mcp_config.json (not found)
Skipping Opencode installation
  Config: C:\Users\orange\.opencode\mcp_config.json (not found)
Skipping Kiro installation
  Config: C:\Users\orange\.kiro\mcp_config.json (not found)
Skipping Trae installation
  Config: C:\Users\orange\.trae\mcp_config.json (not found)
Installed VS Code MCP server (restart required)
```

先知社区

**Trae**

访问下载Trae并安装好

下载地址:https://www.trae.cn/

输入命令查看config

```c
ida-pro-mcp --config
```

```
D:\IDA92\python>"D:\Python311\Scripts\ida-pro-mcp.exe" --config
[HTTP MCP CONFIGURATION]
{
  "mcpServers": {
    "ida-pro-mcp": {
      "type": "http",
      "url": "http://127.0.0.1:13337/mcp"
    }
  }
}

[STDIO MCP CONFIGURATION]
{
  "mcpServers": {
    "ida-pro-mcp": {
      "command": "D:\\Python311\\python.exe",
      "args": [
        "D:\\Python311\\Lib\\site-packages\\ida_pro_mcp\\server.py",
        "--ida-rpc",
        "http://127.0.0.1:13337"
      ]
    }
  }
}
```

先知社区

然后在Trae的设置中找到MCP

把上面的config复制进去

这里要注意的是Trae的路径中不能出现空格，可能是Electron/Node.js 封装的工具在 Windows 上对 spawn 的封装不够健壮，所以可以把带空格的文件夹重命名为无空格的路径。或者转为短路径（Short Path）格式。

复制代码

```c
{
  "mcpServers": {
    "ida-pro-mcp": {
      "command": "D:\\Python311\\python.exe",
      "args": [
        "D:\\Python311\\Lib\\site-packages\\ida_pro_mcp\\server.py",
        "--ida-rpc",
        "http://127.0.0.1:13337"
      ]
    }
  }

```

然后创建智能体

设置 | 智能体 × | MCP

智能体 / IDA

智能生成

名称*

IDA 3/20

提示词

IDAIDAIDA

9/10000

可被其他智能体调用 SOLO Only

可被其他智能体调用

目前仅可以被SOLO模式中的 SOLO Coder 调用。 帮助文档

英文标识名*

IDAIDA 6/50

何时调用*

IDAIDA

6/5000

工具

工具 - MCP

+ 添加 MCP Servers

保存 取消

先知社区

并选中IDA Pro MCP

智能体 / IDA

智能生成

可被其他智能体调用

目前仅可以被SOLO模式中的 SOLO Coder 调用。 帮助文档

英文标识名 *

IDAIDA 6/50

何时调用 *

IDAIDA

6/5000

工具

工具 - MCP

添加 MCP Servers

ida-pro-mcp

工具 - 内置

| | |
|---|---|
| Read | Retrieve and view files |
| Edit | Add, delete and edit files |
| Terminal | Run commands in the terminal and get the status and re... |
| Preview | Provide a preview entry after generating frontend results |
| Web search | Search for web content related to user tasks |

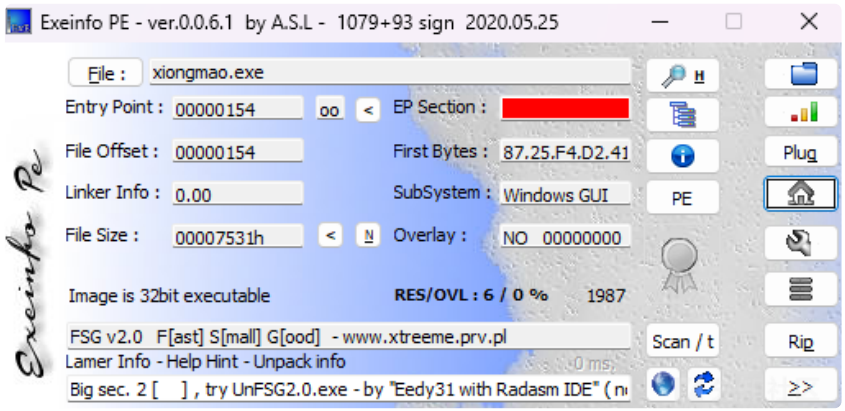
保存 取消

先知社区

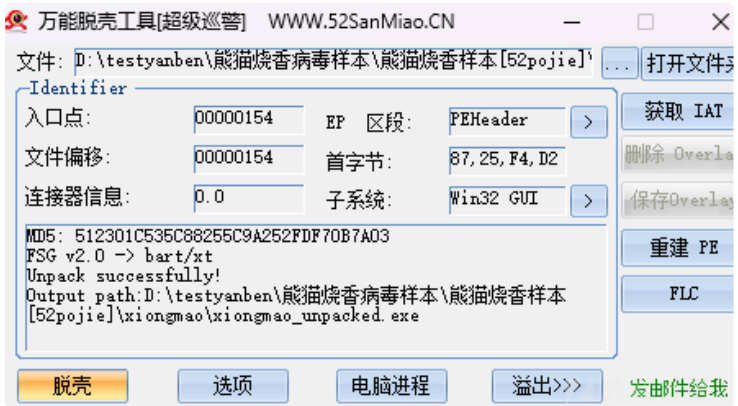
到此，已经基本配置好了。

# 分析过程

注意：在分析之前需要先脱壳，使用的FSG2.0的壳。这也是MCP目前的局限性

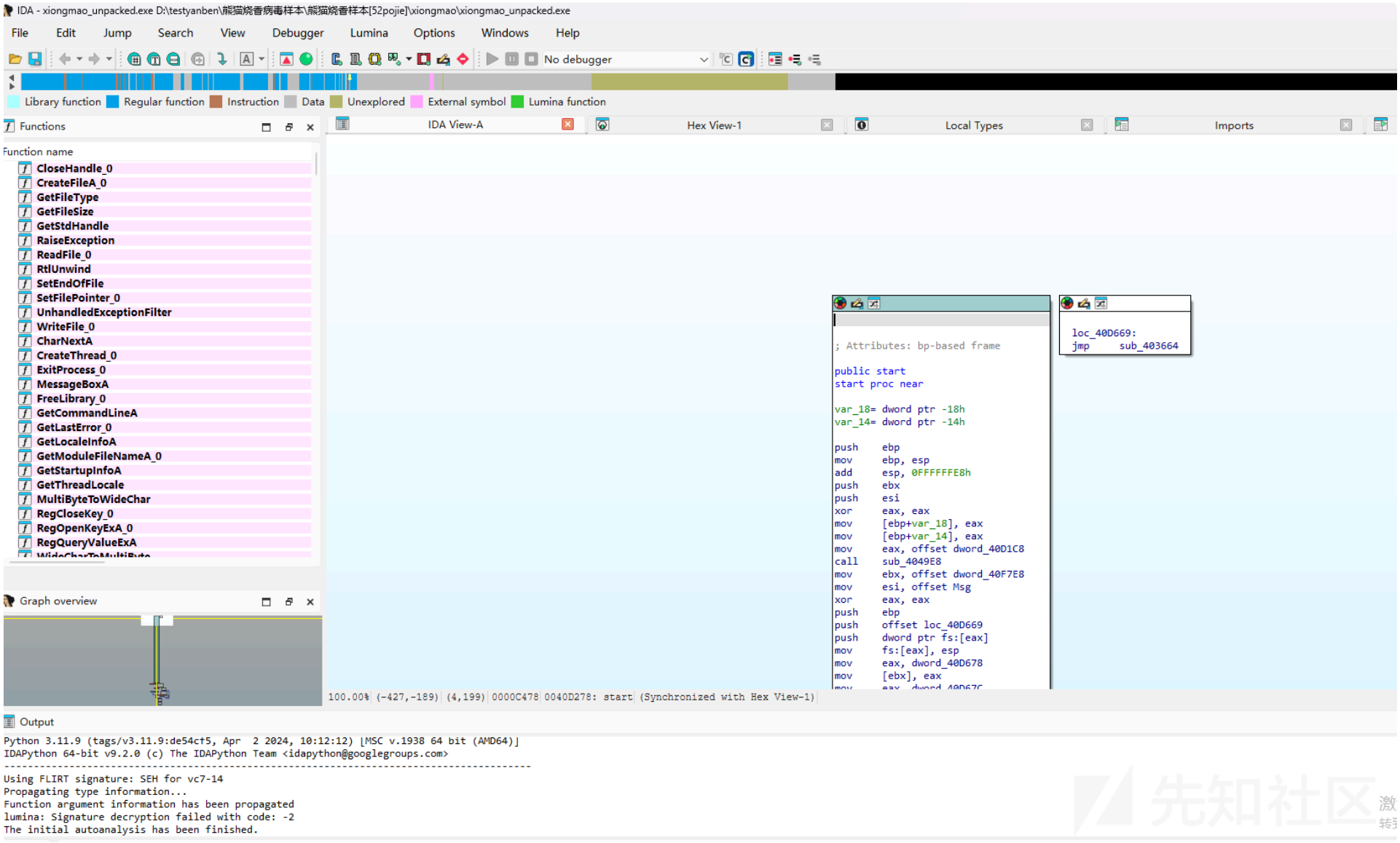



直接掏出脱壳工具进行一键脱壳




运行IDA并加载熊猫烧香病毒程序

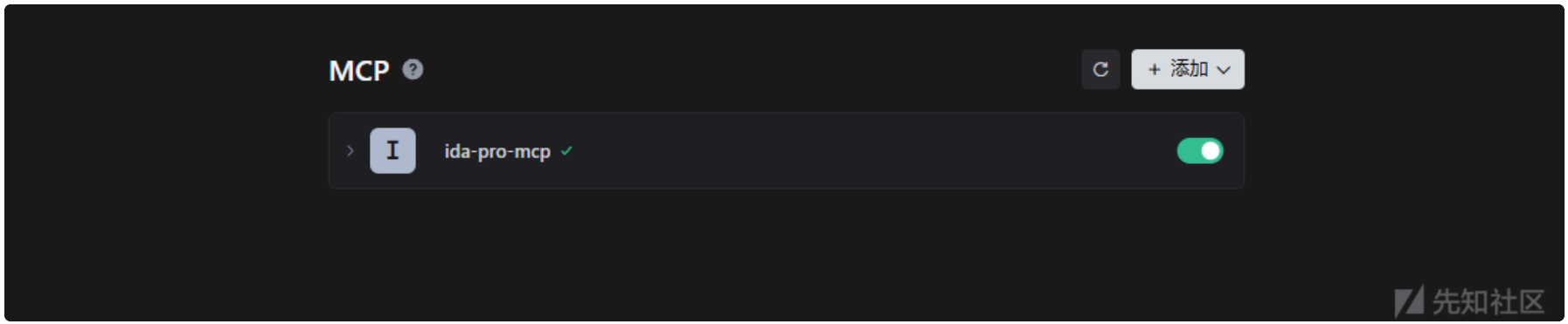
注意：千万不要在物理机上误点击运行了，那么就难受了




在IDA插件功能里启动MCP（IDA打开文件以后，点击左上角Edit--plugins--MCP，你就已经成功打开了MCP服务器，ai可以通过MCP与你的IDA交互了），启动成功如下

```
Output
Function argument information has been propagated
lumina: Signature decryption failed with code: -2
The initial autoanalysis has been finished.
[MCP] Cached 285 strings in 17ms
[MCP] Server started:
  Streamable HTTP: http://127.0.0.1:13337/mcp
  SSE: http://127.0.0.1:13337/sse
  Config: http://127.0.0.1:13337/config.html
Python
```

Trae连接MCP




现在开始就是让MCP进行分析了

输入提示词：分析恶意程序的恶意行为

如下所示，提供输出熊猫烧香的行为操作

分析恶意程序的恶意行为有那些

IDA

思考过程

我来帮你分析这个恶意程序的恶意行为。让我先创建一个分析计划，然后使用IDA Pro工具进行深入分析。

- 列出程序的所有函数，了解程序结构 & 查看导入函数，识别可疑API调用
- 搜索关键字符串，识别恶意行为线索
- 反编译关键函数，分析恶意代码逻辑
- 总结恶意行为特征

分析完成！我已经通过 IDA Pro 对该恶意程序进行了全面的静态分析。以下是该恶意程序的主要恶意行为：

## 恶意行为总结

### 1. 持久化机制

- 修改注册表 Run 键实现开机自启动
- 创建/删除系统服务
- 使用任务调度器（NetScheduleJobAdd）

### 2. 传播方式

- USB 传播：在可移动设备根目录创建 `autorun.inf` 和 `setup.exe`
- 网络共享传播：通过局域网共享传播
- 自我复制到 `drivers\spo0lsv.exe`（伪装成打印后台处理程序）

### 3. 网络恶意行为

- 使用 `URLDownloadToFileA` 从远程服务器下载恶意文件

先知社区

完整的恶意行为如下：

C 复制代码

```c
### 1. 持久化机制
- 修改注册表 Run 键实现开机自启动
- 创建/删除系统服务
- 使用任务调度器（NetScheduleJobAdd）
### 2. 传播方式
- USB 传播 ：在可移动设备根目录创建 autorun.inf 和 setup.exe
- 网络共享传播 ：通过局域网共享传播
- 自我复制到 drivers\spo0lsv.exe （伪装成打印后台处理程序）
### 3. 网络恶意行为
- 使用 URLDownloadToFileA 从远程服务器下载恶意文件
- 使用 InternetOpenUrlA 和 InternetReadFile 进行网络通信
- 检查网络连接状态
### 4. 系统破坏
- 执行命令 cmd.exe /c net share admin$ /del /y 删除网络共享
- 删除系统文件
- 修改文件属性隐藏自身
- 修改注册表隐藏文件显示设置
### 5. 反杀毒软件
- 针对瑞星、McAfee 等多种杀毒软件的注册表启动项进行删除
- 删除杀毒软件相关服务
### 6. 反分析技术
- 使用结构化异常处理（SEH）防止崩溃
- 检测自身进程是否运行
- 文件名伪装（spo0lsv.exe 伪装成 spoolsv.exe）
```

再输入提示词：输出进行每个恶意行为相关的函数以及地址

输出进行每个恶意行为相关的函数以及地址

IDA

思考过程

我将根据之前的分析，整理出每个恶意行为相关的函数及其地址：

整理持久化机制相关函数

检查导入的API & 搜索字符串 & 反编译关键函数 & 总结恶意行为

输出每个恶意行为相关的函数及地址

**恶意行为相关函数及地址映射**

**1. 持久化机制（Persistence）**

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| RegOpenKeyExA | 0x404acc | 打开注册表键 |
| RegSetValueExA | 0x404ad4 | 设置注册表值（Run键持久化） |
| RegDeleteValueA | 0x404ac4 | 删除注册表值 |
| RegCloseKey | 0x404ab4 | 关闭注册表键 |
| NetScheduleJobAdd | 0x40a600 | 添加计划任务 |
| DeleteService | 0x405b6c | 删除服务 |

先知社区

完整结果如下：从结果中可以看到不仅输出了进行恶意行为的函数以及地址，还给了交叉引用的函数。极大的提供了分析效率

## 恶意行为相关函数及地址映射

### 1. 持久化机制（Persistence）

```
### 1. 持久化机制（Persistence）
函数名称 地址 功能描述
RegOpenKeyExA 0x404acc 打开注册表键
RegSetValueExA 0x404ad4 设置注册表值（Run键持久化）
RegDeleteValueA 0x404ac4 删除注册表值
RegCloseKey 0x404ab4 关闭注册表键
NetScheduleJobAdd 0x40a600 添加计划任务
DeleteService 0x405b6c 删除服务
OpenServiceA 0x405b7c 打开服务
sub_4051BC 0x4051bc 注册表Run键写入函数
sub_4058D0 0x4058d0 注册表操作函数
sub_405A50 0x405a50 注册表值删除函数
sub_405C40 0x405c40 服务删除函数
sub_405BBC 0x405bbc 服务操作函数
sub_40A928 0x40a928 计划任务添加函数

关键交叉引用：
- RegSetValueExA 被以下函数调用：sub_4051BC@0x405204, sub_4058D0@0x405983, 0x4059aa
- NetScheduleJobAdd 被调用：sub_40A928@0x40ac07
- DeleteService 被调用：sub_405C40@0x405c8c
```

### 1. 持久化机制（Persistence）

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| RegOpenKeyExA | 0x404acc | 打开注册表键 |
| RegSetValueExA | 0x404ad4 | 设置注册表值（Run键持久化） |
| RegDeleteValueA | 0x404ac4 | 删除注册表值 |
| RegCloseKey | 0x404ab4 | 关闭注册表键 |
| NetScheduleJobAdd | 0x40a600 | 添加计划任务 |
| DeleteService | 0x405b6c | 删除服务 |
| OpenServiceA | 0x405b7c | 打开服务 |
| sub_4051BC | 0x4051bc | 注册表Run键写入函数 |
| sub_4058D0 | 0x4058d0 | 注册表操作函数 |
| sub_405A50 | 0x405a50 | 注册表值删除函数 |
| sub_405C40 | 0x405c40 | 服务删除函数 |

先知社区

### 2. 传播机制（Propagation）

```c
### 2. 传播机制（Propagation）
函数名称 地址 功能描述
TimerFunc 0x40bf9f 定时器函数（USB传播）
sub_402DD8 0x402dd8 文件创建函数
sub_402F40 0x402f40 字符串解析函数
sub_4037D8 0x4037d8 传播相关函数（反编译失败）

关键字符串引用：
- "autorun.inf" 在 TimerFunc 中的地址：0x40c308, 0x40c320
- "setup.exe" 在 TimerFunc 中的地址：0x40c2f4
```

**2. 传播机制（Propagation）**

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| TimerFunc | 0x40bf9f | 定时器函数（USB传播） |
| sub_402DD8 | 0x402dd8 | 文件创建函数 |
| sub_402F40 | 0x402f40 | 字符串解析函数 |
| sub_4037D8 | 0x4037d8 | 传播相关函数（反编译失败） |

**关键字符串引用：**

- "autorun.inf" 在 TimerFunc 中的地址：0x40c308, 0x40c320
- "setup.exe" 在 TimerFunc 中的地址：0x40c2f4

先知社区

### 3. 网络恶意行为（Network Activity）

C 复制代码

```c
### 3. 网络恶意行为（Network Activity）
函数名称 地址 功能描述
URLDownloadToFileA 0x40c4e4 下载文件到本地
InternetOpenUrlA 0x404d5c 打开URL连接
InternetReadFile 0x404d64 读取网络数据
InternetGetConnectedState 0x404d6c 检查网络连接状态
sub_40C5E0 0x40c5e0 网络下载函数
sub_40C728 0x40c728 网络操作函数
sub_40C9B0 0x40c9b0 网络通信函数
sub_40B864 0x40b864 网络状态检查函数

关键交叉引用：
- URLDownloadToFileA 被调用：sub_40C728@0x40c832, 0x40c8cd; sub_40C9B0@0x40caba, 0x40cb55
- InternetOpenUrlA 被调用：sub_40C5E0@0x40c651
- InternetReadFile 被调用：sub_40C5E0@0x40c696
- InternetGetConnectedState 被调用：sub_40B864@0x40b89e
```

### 3. 网络恶意行为（Network Activity）

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| URLDownloadToFileA | 0x40c4e4 | 下载文件到本地 |
| InternetOpenUrlA | 0x404d5c | 打开URL连接 |
| InternetReadFile | 0x404d64 | 读取网络数据 |
| InternetGetConnectedState | 0x404d6c | 检查网络连接状态 |
| sub_40C5E0 | 0x40c5e0 | 网络下载函数 |
| sub_40C728 | 0x40c728 | 网络操作函数 |
| sub_40C9B0 | 0x40c9b0 | 网络通信函数 |
| sub_40B864 | 0x40b864 | 网络状态检查函数 |

**关键交叉引用：**

- URLDownloadToFileA 被调用：sub_40C728@0x40c832, 0x40c8cd; sub_40C9B0@0x40caba, 0x40cb55
- InternetOpenUrlA 被调用：sub_40C5E0@0x40c651

先知社区

### 4．系统破坏（System Destruction）

```c
### 4. 系统破坏（System Destruction）
函数名称 地址 功能描述
WinExec 0x404bd4 执行系统命令
sub_40CDEC 0x40cdec 执行删除网络共享命令
sub_407B68 0x407b68 系统命令执行函数
sub_40819C 0x40819c 系统破坏函数

关键交叉引用：

- WinExec 被调用：sub_407B68@0x407e06, sub_40819C@0x408452, 0x408721, sub_40C728@0x40c874, 0x40c90f, sub_40C9B0@0x40cafc, 0x40cb97
- sub_40CDEC 在 0x40ce4f 处执行命令："cmd.exe /c net share admin$ /del /y"
```

### 4. 系统破坏（System Destruction）

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| WinExec | 0x404bd4 | 执行系统命令 |
| sub_40CDEC | 0x40cdec | 执行删除网络共享命令 |
| sub_407B68 | 0x407b68 | 系统命令执行函数 |
| sub_40819C | 0x40819c | 系统破坏函数 |

**关键交叉引用：**

- WinExec 被调用：sub_407B68@0x407e06, sub_40819C@0x408452, 0x408721, sub_40C728@0x40c874, 0x40c90f, sub_40C9B0@0x40cafc, 0x40cb97, sub_40CDEC@0x40ce4f, 0x40ce60
- sub_40CDEC 在 0x40ce4f 处执行命令："cmd.exe /c net share admin$ /del /y"

先知社区

## 5. 反杀毒软件（Anti-Antivirus）

```c
### 5. 反杀毒软件（Anti-Antivirus）
函数名称 地址 功能描述 sub_406E44 0x406e44 删除杀毒软件注册表键

目标杀毒软件注册表键：

- RavTask
- McAfeeUpdaterUI
- 其他杀毒软件相关键值
```

### 5. 反杀毒软件（Anti-Antivirus）

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| sub_406E44 | 0x406e44 | 删除杀毒软件注册表键 |

**目标杀毒软件注册表键：**

- RavTask
- McAfeeUpdaterUI
- 其他杀毒软件相关键值

先知社区

## 6. 反分析技术（Anti-Analysis）

```c
### 6. 反分析技术（Anti-Analysis）
函数名称 地址 功能描述
sub_4015A0 0x4015a0 内存管理函数（可能用于反调试）
sub_402280 0x402280 数据处理函数（可能用于代码混淆）
```

## 7. 文件操作（File Operations）

C 复制代码

```
### 7. 文件操作（File Operations）
函数名称 地址 功能描述
CreateFileA - 创建文件（导入API）
ReadFile - 读取文件（导入API）
WriteFile - 写入文件（导入API）
FindFirstFileA - 查找文件（导入API）
FindNextFileA - 查找下一个文件（导入API）
FindClose - 关闭查找句柄（导入API）

### 关键字符串及地址
字符串 地址 用途
"spo0lsv.exe" - 伪装文件名
"setup.exe" 0x40c2f4 USB传播文件
"autorun.inf" 0x40c308, 0x40c320 USB自动运行配置
"admin$" - 管理共享
"cmd.exe /c net share admin$ /del /y" 0x40ce4f 删除管理共享命令
"SOFTWARE\Microsoft\Windows\CurrentVersion\Run" - 注册表启动项路径

### 注册表操作相关路径
路径 用途
SOFTWARE\Microsoft\Windows\CurrentVersion\Run 启动项持久化 杀毒软件相关注册表键 反杀毒软件


```

### 7. 文件操作（File Operations）

| 函数名称 | 地址 | 功能描述 |
| --- | --- | --- |
| CreateFileA | - | 创建文件（导入API） |
| ReadFile | - | 读取文件（导入API） |
| WriteFile | - | 写入文件（导入API） |
| FindFirstFileA | - | 查找文件（导入API） |
| FindNextFileA | - | 查找下一个文件（导入API） |
| FindClose | - | 关闭查找句柄（导入API） |

### 关键字符串及地址

| 字符串 | 地址 | 用途 |
| --- | --- | --- |
| "spo0lsv.exe" | - | 伪装文件名 |
| "setup.exe" | 0x40c2f4 | USB传播文件 |

先知社区

甚至能够协助获取到制作者遗留在病毒程序中的信息

## 制作者信息分析

### 1. 制作者标识符

在程序中发现了 **"WhBoy"** 和 **"whboy"** 字符串，这很可能是制作者的昵称或代号：

| 地址 | 字符串 | 位置 |
|---|---|---|
| 0x40815c | "WhBoy" | 数据段 |
| 0x40d8cc | "whboy" | 数据段 |
| 0x41a720 | "WhBoy" | 数据段 |

### 2. 制作者标识符的使用

**在 start 函数（地址 0x40d600）中：**

Assembly

```
mov     edx, offset aWhboy_0; "whboy"
mov     eax, offset unk_40D8DC
call    sub_405250
```

- "whboy" 被用作病毒标记，用于检查病毒是否已经在运行
- 如果标记存在，程序会退出，防止重复感染

**在 sub_407F00 函数（地址 0x407f00）中：**

- "WhBoy" 也被引用，可能用于其他检查或标识目的

### 3. 制作者信息总结

**找到的信息：**

- ✅ 制作者昵称/代号：WhBoy（武汉男孩）
- ✅ 病毒标记：使用 "whboy" 作为运行标识

先知社区

## 具体函数分析

继续让MCP逐一分析各个函数。

## sub_402DD8函数

提示词：提供sub_402DD8函数伪代码以及注释，说明

MCP反馈sub_402DD8 是一个 文件创建/打开函数 ， 用于创建或打开文件，也可以获取标准输入/输出句柄。

- 文件创建：用于创建病毒副本、配置文件（如 autorun.inf ）或其他恶意文件

- 持久化：可能用于创建启动脚本或配置文件以实现自启动

- 传播：结合 autorun.inf 和 setup.exe 字符串，用于在USB驱动器上创建传播文件

- 日志记录：可能使用标准输出句柄记录病毒活动

提供sub_402DD8函数伪代码以及注释，说明

IDA

思考过程

ida-pro-mcp/decompile

Thought

## sub_402DD8 函数伪代码及注释说明

C

```c
HANDLE __usercall sub_402DD8@<eax>(int a1@<eax>, int a2@<edx>, int a3@<ecx>)
{
  DWORD v6;           // 文件创建标志
  DWORD v7;           // 文件共享模式
  DWORD v8;           // 文件创建 disposition
  int v9;             // 临时变量
  int v10;            // 临时变量
  HANDLE result;      // 返回的文件句柄

  // 检查状态标志是否为 0xD780 (55216)
  if ( *(unsigned __int16 *)(a1 + 4) != 55216 )
  {
    // 如果状态标志不在 55216-55219 范围内，调用错误处理函数
    if ( (unsigned int)*(unsigned __int16 *)(a1 + 4) - 55216 > 3 )
      return (HANDLE)sub_402634();
    // 如果存在回调函数，先调用它再进行错误处理
    if ( (*(int (**)(void))(a1 + 36))() )
      sub_402634();
  }

  // 设置状态标志为 0xD7A3 (-10317)
  *(_WORD *)(a1 + 4) = -10317;

  // 保存参数 a2 到偏移 +8 处
  *(_DWORD *)(a1 + 8) = a2;
```

先知社区

sub_402DD8 C 复制代码

```c
HANDLE __usercall sub_402DD8@<eax>(int a1@<eax>, int a2@<edx>, int a3@<ecx>)
{
  DWORD v6;           // 文件创建标志
  DWORD v7;           // 文件共享模式
  DWORD v8;           // 文件创建 disposition
  int v9;             // 临时变量
  int v10;            // 临时变量
  HANDLE result;      // 返回的文件句柄

  // 检查状态标志是否为 0xD780 (55216)
  if ( *(unsigned __int16 *)(a1 + 4) != 55216 )
  {
    // 如果状态标志不在 55216-55219 范围内，调用错误处理函数
    if ( (unsigned int)*(unsigned __int16 *)(a1 + 4) - 55216 > 3 )
      return (HANDLE)sub_402634();
    // 如果存在回调函数，先调用它再进行错误处理
    if ( (*(int (**)(void))(a1 + 36))() )
      sub_402634();
  }

  // 设置状态标志为 0xD7A3 (-10317)
  *(_WORD *)(a1 + 4) = -10317;

  // 保存参数 a2 到偏移 +8 处
  *(_DWORD *)(a1 + 8) = a2;

  // 设置回调函数指针
  *(_DWORD *)(a1 + 36) = sub_402DB0;
  *(_DWORD *)(a1 + 28) = sub_4028BC;

  // 检查是否有文件名（偏移 +72 处）
  if ( *(_BYTE *)(a1 + 72) )
  {
    // 初始化文件创建参数
    v6 = -1073741824;  // GENERIC_READ (0x80000000)

```

```
37        // 从全局数据获取共享模式
38        v7 = *(int *)((char *)&dword_40E058 + ((unsigned __int8)(byte_40E00C & 0x70) >> 2));
39
40        v8 = 2;  // OPEN_EXISTING
41        v9 = a3 - 3;
42
43        // 根据 a3 参数确定文件创建方式
44        if ( v9 )
45        {
46          v8 = 3;  // OPEN_ALWAYS
47          v10 = v9 + 1;
48          if ( v10 )
49          {
50            v6 = 0x40000000;  // GENERIC_WRITE
51            *(_WORD *)(a1 + 4) = -10318;
52            if ( v10 != -1 )
53            {
54              v6 = 0x80000000;  // GENERIC_READ
55              *(_WORD *)(a1 + 4) = -10319;
56            }
57          }
58        }
59
60        // 创建文件
61        // 参数：文件名、访问权限、共享模式、安全属性、创建方式、文件属性、模板文件
62        result = CreateFileA_0((LPCSTR)(a1 + 72), v6, v7, 0, v8, 0x80u, 0);
63      }
64      else
65      {
66        // 如果没有文件名，使用标准输入/输出句柄
67        *(_DWORD *)(a1 + 36) = sub_4028BC;
68        if ( a3 == 3 )
69          result = GetStdHandle(0xFFFFFFF5);  // STD_OUTPUT_HANDLE
70        else
71          result = GetStdHandle(0xFFFFFFF6);  // STD_ERROR_HANDLE
72      }
73
74      // 检查文件句柄是否有效
75      if ( result != (HANDLE)-1 )
```

```
  {
    // 保存文件句柄到结构体
    *(_DWORD *)a1 = result;
    return result;
  }

  // 文件创建失败，设置错误状态
  *(_WORD *)(a1 + 4) = -10320;
  GetLastError_0();
  return (HANDLE)sub_402634();
```

进入IDA验证是否一致，可以看到基本一致。

```
HANDLE __usercall sub_402DD8@<eax>(int a1@<eax>, int a2@<edx>, int a3@<ecx>)
{
  DWORD v6; // eax
  DWORD v7; // edx
  DWORD v8; // ecx
  int v9; // edi
  int v10; // edi
  HANDLE result; // eax

  if ( *(unsigned __int16 *)(a1 + 4) != 55216 )
  {
    if ( (unsigned int)*(unsigned __int16 *)(a1 + 4) - 55216 > 3 )
      return (HANDLE)sub_402634();
    if ( (*(int (**)(void))(a1 + 36))() )
      sub_402634();
  }
  *(_WORD *)(a1 + 4) = -10317;
  *(_DWORD *)(a1 + 8) = a2;
  *(_DWORD *)(a1 + 36) = sub_402DB0;
  *(_DWORD *)(a1 + 28) = sub_4028BC;
  if ( *(_BYTE *)(a1 + 72) )
  {
    v6 = -1073741824;
    v7 = *(int *)((char *)&dword_40E058 + ((unsigned __int8)(byte_40E00C & 0x70) >> 2));
    v8 = 2;
    v9 = a3 - 3;
    if ( v9 )
    {
      v8 = 3;
      v10 = v9 + 1;
      if ( v10 )
      {
        v6 = 0x40000000;
        *(_WORD *)(a1 + 4) = -10318;
        if ( v10 != -1 )
        {
          v6 = 0x80000000;
          *(_WORD *)(a1 + 4) = -10319;
        }
      }
    }
    result = CreateFileA_0((LPCSTR)(a1 + 72), v6, v7, 0, v8, 0x80u, 0);
  }
  else
  {
    *(_DWORD *)(a1 + 36) = sub_4028BC;
    if ( a3 == 3 )
```

先知社区

再分析一下其他几个相对重要的函数

**sub_40C5E0函数**

sub_40C5E0 是一个 网络下载函数 ， 用于从指定的URL下载文件内容到内存缓冲区。

执行流程如下:

```c
### 执行流程
1. 初始化异常处理框架 : 设置SEH（结构化异常处理）以捕获可能的异常
2. 初始化网络会话 :

   - 调用 InternetOpenA 初始化WinINet会话
   - 使用默认用户代理字符串（ szAgent ）
3. 打开URL连接 :

   - 调用 InternetOpenUrlA 打开指定的URL
   - 使用标志 0x84000002 :
     - INTERNET_FLAG_RELOAD : 强制从服务器重新下载，不使用缓存
     - INTERNET_FLAG_NO_CACHE_WRITE : 不将下载的数据写入缓存
4. 下载数据 :

   - 循环调用 InternetReadFile 读取数据
   - 每次最多读取 1024 字节（0x400）
   - 将读取的数据保存到缓冲区
   - 通过 sub_403EB4 和 sub_403ED4 处理数据（可能是写入文件或内存）
5. 清理资源 :

   - 关闭URL句柄和Internet句柄
   - 恢复异常处理链
```

提供sub_40C5E0函数伪代码以及注释，用途

IDA

思考过程

ida-pro-mcp/decompile

Thought

## sub_40C5E0 函数伪代码及注释说明

C

```c
int __usercall sub_40C5E0@<eax>(const CHAR *a1@<eax>)
{
  void *v2;                    // Internet句柄
  void *v3;                    // Internet句柄备份
  void *v4;                    // URL句柄
  unsigned int v6[2];          // 异常处理链
  int *v7;                     // 保存的寄存器
  int v8;                      // 标志变量
  _BYTE Buffer[1025];          // 读取缓冲区（1024字节 + 1字节空终止符）
  DWORD dwNumberOfBytesRead[2]; // 读取的字节数
  int savedregs;               // 保存的寄存器

  v8 = 0;
  dwNumberOfBytesRead[1] = 0;
  v7 = &savedregs;

  // 设置异常处理框架
  v6[1] = (unsigned int)&loc_40C708;
  v6[0] = (unsigned int)NtCurrentTeb()->NtTib.ExceptionList;
  __writefsdword(0, (unsigned int)v6);

  // 初始化函数
  sub_403CDC();
```

先知社区

C 复制代码

```c
int __usercall sub_40C5E0@<eax>(const CHAR *a1@<eax>)
{
  void *v2;                    // Internet句柄
  void *v3;                    // Internet句柄备份
  void *v4;                    // URL句柄
  unsigned int v6[2];          // 异常处理链
  int *v7;                     // 保存的寄存器
  int v8;                      // 标志变量
  _BYTE Buffer[1025];          // 读取缓冲区（1024字节 + 1字节空终止符）
  DWORD dwNumberOfBytesRead[2]; // 读取的字节数
  int savedregs;              // 保存的寄存器

  v8 = 0;
  dwNumberOfBytesRead[1] = 0;
  v7 = &savedregs;

  // 设置异常处理框架
  v6[1] = (unsigned int)&loc_40C708;
  v6[0] = (unsigned int)NtCurrentTeb()->NtTib.ExceptionList;
  __writefsdword(0, (unsigned int)v6);

  // 初始化函数
  sub_403CDC();
  sub_403C44(v6[0]);

  // 打开Internet会话
  // 参数：用户代理字符串、访问类型、代理服务器、代理绕过列表、标志
  v2 = InternetOpenA(szAgent, 0, 0, 0, 0);
  v3 = v2;

  if ( v2 )
  {
    // 打开URL连接
    // 参数：Internet句柄、URL、HTTP头、HTTP头长度、标志、上下文
    // 标志 0x84000002 = INTERNET_FLAG_RELOAD | INTERNET_FLAG_NO_CACHE_WRITE
    v4 = InternetOpenUrlA(v2, a1, 0, 0, 0x84000002, 0);
```

```
37
38      if ( v4 )
39      {
40        // 初始化缓冲区（1025字节）
41        sub_402D74(0, 1025);
42
43        // 循环读取网络数据
44        while ( 1 )
45        {
46          while ( 1 )
47          {
48            dwNumberOfBytesRead[0] = 0;
49
50            // 从URL读取数据
51            // 参数：URL句柄、缓冲区、缓冲区大小、实际读取字节数
52            if ( InternetReadFile(v4, Buffer, 0x400u, dwNumberOfBytesRead) )
53              break;
54
55            // 读取失败，调用错误处理
56            sub_403C98();
57          }
58
59          // 如果没有更多数据，退出循环
60          if ( !dwNumberOfBytesRead[0] )
61            break;
62
63          // 添加字符串结束符
64          Buffer[dwNumberOfBytesRead[0]] = 0;
65
66          // 处理读取的数据（可能是保存到缓冲区或文件）
67          sub_403EB4(1025, Buffer);
68          sub_403ED4();
69        }
70
71        // 关闭URL句柄
72        InternetCloseHandle(v3);
73      }
74      else
75      {
```

```
      // URL打开失败，关闭Internet句柄
      InternetCloseHandle(v3);
      sub_403C98();
    }
  }
  else
  {
    // Internet会话创建失败
    sub_403C98();
  }

  // 恢复异常处理链
  __writefsdword(0, v6[0]);
  sub_403C44(&loc_40C70F);
  return sub_403C44(v7);
```

**sub_40CDEC函数**

sub_40CDEC 是一个 系统破坏函数 ， 用于执行破坏性系统命令，删除网络共享并可能进行其他系统破坏操作。

主要执行的命令是：`cmd.exe /c net share admin$ /del /y`

C 复制代码

```
### 执行流程
1. 设置异常处理框架 ： 使用SEH（结构化异常处理）捕获可能的异常
2. 初始化 ： 调用 sub_4075A4() 进行初始化，获取循环次数
3. 循环执行破坏命令 ：

   - 调用 sub_403E2C() 准备命令参数
   - 调用 sub_403F8C() 构建命令字符串
   - 调用 sub_4040CC(0) 获取完整的命令字符串
   - 使用 WinExec() 执行命令
   - 根据循环次数重复执行
4. 执行后续命令 ： 调用 WinExec(CmdLine, 0) 执行另一个命令
5. 清理资源 ： 恢复异常处理链
```

sub_40CDEC函数的伪代码以及注释，用途

IDA

思考过程

ida-pro-mcp/decompile

Thought

## sub_40CDEC 函数伪代码及注释说明

C

```c
int __usercall sub_40CDEC@<eax>(unsigned int a1@<ebx>)
{
  int v1;                    // 循环计数器
  const CHAR *v2;            // 命令字符串指针
  UINT v4;                   // WinExec显示模式参数
  unsigned int v5[4];        // 异常处理链
  int v6;                    // 临时变量
  int v7;                    // 临时变量
  int v8;                    // 标志变量
  int savedregs;            // 保存的寄存器

  v8 = 0;
  v7 = 0;
  v6 = 0;

  // 保存参数到异常处理结构
  v5[3] = a1;
  v5[2] = (unsigned int)&savedregs;
  v5[1] = (unsigned int)&loc_40CE80;
  v5[0] = (unsigned int)NtCurrentTeb()->NtTib.ExceptionList;

```

先知社区

C 复制代码

```c
int __usercall sub_40CDEC@<eax>(unsigned int a1@<ebx>)
{
  int v1;                     // 循环计数器
  const CHAR *v2;            // 命令字符串指针
  UINT v4;                   // WinExec显示模式参数
  unsigned int v5[4];        // 异常处理链
  int v6;                    // 临时变量
  int v7;                    // 临时变量
  int v8;                    // 标志变量
  int savedregs;            // 保存的寄存器

  v8 = 0;
  v7 = 0;
  v6 = 0;

  // 保存参数到异常处理结构
  v5[3] = a1;
  v5[2] = (unsigned int)&savedregs;
  v5[1] = (unsigned int)&loc_40CE80;
  v5[0] = (unsigned int)NtCurrentTeb()->NtTib.ExceptionList;

  // 设置异常处理框架
  __writefsdword(0, (unsigned int)v5);

  // 初始化函数
  sub_4075A4();
  v1 = sub_403ECC(v5[0]);

  // 循环执行系统破坏命令
  if ( v1 >= 1 )
  {
    do
    {
      // 准备命令参数
      sub_403E2C();

```

```
        // 构建命令字符串
        // 参数: "aDelY" (可能是 "/del" 或类似参数), v6, "aCmdExeCNetShar" (可能是 "cmd.exe /c net share")
        sub_403F8C(aDelY, v6, aCmdExeCNetShar);

        // 获取完整的命令字符串
        v2 = (const CHAR *)sub_4040CC(0);

        // 执行系统命令
        // 实际执行的命令: "cmd.exe /c net share admin$ /del /y"
        WinExec(v2, v4);

        --v1;
      }
      while ( v1 );
    }

    // 执行另一个命令（可能是清理或后续破坏）
    WinExec(CmdLine, 0);

    // 恢复异常处理链
    __writefsdword(0, v5[0]);
    return sub_403C68(&loc_40CE87);
  }
```

**sub_406E44函数**

sub_406E44 是一个 反杀毒软件函数 ， 用于删除多种杀毒软件的服务和注册表启动项，使杀毒软件失效。

执行流程如下：

C 复制代码

```
### 执行流程
该函数按顺序针对以下杀毒软件进行攻击：

1. 瑞星杀毒软件 :

   - 删除 4 个服务（ sub_405BBC ）
   - 删除 2 个服务（ sub_405C40 ）
   - 删除注册表启动项： RavTask
2. 江民杀毒软件 :

   - 删除 2 个服务（ sub_405BBC ）
   - 删除 2 个服务（ sub_405C40 ）
   - 删除注册表启动项： KvMonXP
3. 卡巴斯基杀毒软件 :

   - 删除 2 个服务（ sub_405BBC ）
   - 删除 2 个服务（ sub_405C40 ）
   - 删除注册表启动项： kav
   - 删除注册表启动项： KAVPersonal50
4. McAfee杀毒软件 :

   - 删除 3 个服务（ sub_405BBC ）
   - 删除 3 个服务（ sub_405C40 ）
   - 删除注册表启动项： McAfeeUpdaterUI
   - 删除注册表启动项： Network Associates Error Reporting Service
   - 删除注册表启动项： ShStatEXE
5. 金山毒霸 :

   - 删除 12 个服务（ sub_405C40 ）
   - 删除注册表启动项： YLive.exe
   - 删除注册表启动项： yassistse
```

**sub_406E44 函数伪代码及注释说明**

```c
int __stdcall sub_406E44()
{
  // 第一组：删除瑞星杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除瑞星杀毒软件注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\RavTask"
  sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_1);

  // 第二组：删除江民杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除江民杀毒软件注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\KvMonXP"
  sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_2);

  // 第三组：删除卡巴斯基杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除卡巴斯基杀毒软件注册表启动项
```

先知社区

C 复制代码

```c
int __stdcall sub_406E44()
{
  // 第一组：删除瑞星杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除瑞星杀毒软件注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\RavTask"
  sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_1);

  // 第二组：删除江民杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除江民杀毒软件注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\KvMonXP"
  sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_2);

  // 第三组：删除卡巴斯基杀毒软件相关服务和注册表项
  sub_405BBC();  // 删除服务
  sub_405BBC();  // 删除服务
  sub_405C40();  // 删除服务
  sub_405C40();  // 删除服务

  // 删除卡巴斯基杀毒软件注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\kav"
  sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_3);

  // 删除卡巴斯基个人版注册表启动项
  // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\KAVPersonal50"
```

```
37      sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_4);
38
39      // 第四组：删除McAfee杀毒软件相关服务和注册表项
40      sub_405BBC();  // 删除服务
41      sub_405BBC();  // 删除服务
42      sub_405BBC();  // 删除服务
43      sub_405C40();  // 删除服务
44      sub_405C40();  // 删除服务
45      sub_405C40();  // 删除服务
46
47      // 删除McAfee杀毒软件注册表启动项
48      // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\McAfeeUpdaterUI"
49      sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_5);
50
51      // 删除McAfee错误报告服务注册表启动项
52      // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\Network Associates Error Reporting Service"
53      sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_6);
54
55      // 删除McAfee状态监控程序注册表启动项
56      // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\ShStatEXE"
57      sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_7);
58
59      // 第五组：删除金山毒霸相关服务和注册表项
60      sub_405C40();  // 删除服务
61      sub_405C40();  // 删除服务
62      sub_405C40();  // 删除服务
63      sub_405C40();  // 删除服务
64      sub_405C40();  // 删除服务
65      sub_405C40();  // 删除服务
66      sub_405C40();  // 删除服务
67      sub_405C40();  // 删除服务
68      sub_405C40();  // 删除服务
69      sub_405C40();  // 删除服务
70      sub_405C40();  // 删除服务
71      sub_405C40();  // 删除服务
72
73      // 删除金山毒霸注册表启动项
74      // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\YLive.exe"
75      sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_8);
```

```
76
77     // 删除金山毒霸辅助服务注册表启动项
78     // "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\\yassistse"
79     return sub_405A50(HKEY_LOCAL_MACHINE, (int)aSoftwareMicros_9);
```

## 病毒程序运行流程

病毒程序的运行流程如下：

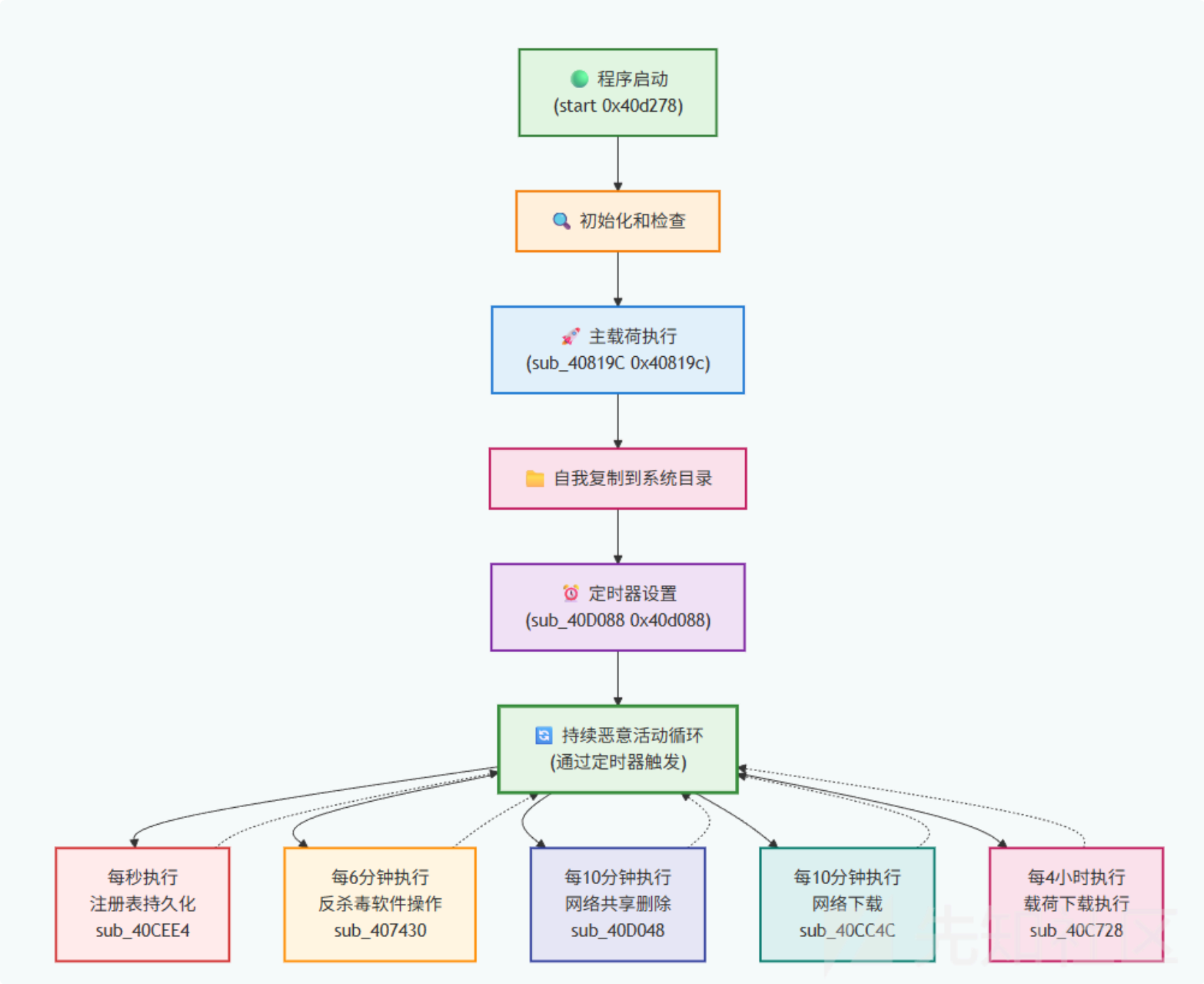
C 复制代码

```c
### 第一阶段：程序初始化与启动 1. 入口点 - start函数 (地址: 0x40d278)
作用： 程序启动的入口，负责初始化和启动恶意载荷

执行步骤：

- 设置异常处理机制
- 初始化全局数据结构和内存
- 检查病毒标记（"Xboy" 和 "Whboy" 字符串）
- 如果标记不存在，调用主载荷执行函数
- 调用定时器设置函数
- 进入消息循环，保持程序运行
### 第二阶段：主载荷执行 2. 主载荷执行 - sub_40819C函数 (地址: 0x40819c)
作用： 病毒的核心感染和自我复制功能

执行步骤：

- 检查病毒是否已经在运行
- 如果已运行，删除自身文件并退出
- 如果未运行：
  - 将自身复制到系统目录 %SystemRoot%\drivers\spo0lsv.exe
  - 设置文件属性为隐藏和系统文件
  - 执行复制的文件
  - 退出当前进程
恶意行为： 实现病毒的持久化和自我复制

### 第三阶段：定时器设置与持续恶意活动 3. 定时器设置 - sub_40D088函数 (地址: 0x40d088)
作用： 设置多个定时器，实现病毒的持续恶意活动

### 第四阶段：定时器回调函数执行 4. 注册表持久化 - sub_40CEE4函数 (地址: 0x40cee4)
执行间隔： 每1秒执行一次

作用： 确保病毒在系统重启后自动启动

执行步骤：

- 在注册表 HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run 中添加启动项
```

```
- 启动项名称为 "Svcshare"
- 指向病毒文件路径
恶意行为： 实现开机自启动

5. 网络共享删除 - sub_40D048函数 (地址: 0x40d048)
执行间隔： 每10分钟执行一次

作用： 破坏系统网络共享功能

执行步骤：

- 创建线程执行 sub_40CC34（功能未知）
- 创建线程执行 sub_40CDEC（系统破坏）
- 关闭定时器
恶意行为： 删除系统网络共享，影响网络功能

6. 反杀毒软件 - sub_407430函数 (地址: 0x407430)
执行间隔： 每6分钟执行一次

作用： 禁用杀毒软件，逃避检测

执行步骤：

- 创建线程执行 sub_406E44
- sub_406E44 删除多种杀毒软件的注册表启动项
恶意行为： 禁用瑞星、McAfee、卡巴斯基等杀毒软件
 7. 网络下载 - sub_40CC4C函数 (地址: 0x40cc4c)
执行间隔： 每10分钟执行一次

作用： 从远程服务器下载恶意文件

执行步骤：

- 构造多个下载URL
- 调用 sub_40C5E0 下载文件
- 下载多个恶意文件到本地
恶意行为： 下载并执行额外的恶意载荷
 8. 载荷下载与执行 - sub_40C728函数 (地址: 0x40c728)
执行间隔： 每4小时执行一次
```

```

作用： 下载并执行高级恶意载荷

执行步骤：

- 构造下载URL
- 使用 URLDownloadToFileA API 下载文件
- 使用 WinExec API 执行下载的文件
- 循环下载和执行多个文件
恶意行为： 下载并执行额外的恶意程序，可能包括：

- 更新病毒
- 下载其他恶意软件
```

完整执行流程图如下：


程序启动
(start 0x40d278)
初始化和检查
主载荷执行
(sub_40819C 0x40819c)
自我复制到系统目录
定时器设置
(sub_40D088 0x40d088)
持续恶意活动循环
(通过定时器触发)
每秒执行
注册表持久化
sub_40CEE4
每6分钟执行
反杀毒软件操作
sub_407430
每10分钟执行
网络共享删除
sub_40D048
每10分钟执行
网络下载
sub_40CC4C
每4小时执行
载荷下载执行
sub_40C728

**值得注意的是IDA Pro MCP并不能很好的分析从内存中释放的恶意载荷，因此还需要人工去动态调试这一部分**

0 人收藏 0 人喜欢 转载 分享

0 条评论